Received XX Month 2026, accepted XX Month 2026, date of publication XX Month 2026, date of current version XX Month 2026.
Digital Object Identifier 10.1109/ACCESS.2026.XXXXXXX

An 8% Learned Energy Saving That Was a Plant-Model Artefact: Processor-in-the-Loop Evaluation of Rule-Based and Learned Electric-Vehicle Energy Management on a Microcontroller

MANIKANTA GONUGONDLA[1] AND [CO-AUTHOR NAMES TO BE CONFIRMED][1]

1[Department and institution to be confirmed]

Corresponding author: Manikanta Gonugondla (e-mail to be confirmed).

MOCK MANUSCRIPT prepared for internal review. Not submitted to any journal. Dates, volume and Digital Object Identifier are placeholders.

ABSTRACT Learned energy managers for electric vehicles are mostly evaluated in simulation, with the vehicle model taken on trust. In a processor-in-the-loop study, a Soft Actor-Critic policy running on an ESP32-S3 microcontroller at 100 Hz appeared to use 8.28% less energy than a rule-based manager. The gain came from the vehicle model, which billed battery power at the policy's chosen ceiling but computed acceleration from the uncapped torque; a sweep with no controller in the loop reproduced it. The validation gate on logged trips never exercised that path. Anomalies in the campaign tables, not the gates, triggered the audit, and three checks that would have caught it earlier are given. After the correction the learned policy used more energy than the rules in 11 of 12 distinct conditions on the target, by a median of 0.31% (95% interval 0.17% to 0.76%). A dynamic-programming bound explains the null result: with speed fixed by the tracking loop, the rules sit within 0.16% of the optimum. The measured gain lay in the speed loop instead. A two-state Kalman speed estimator cut the energy lost at ten times the sensor-noise floor from 9.44 to 0.24 Wh/km on the target, and learned fault reviewers that classified better offline controlled worse in the loop in simulation. A Transformer power forecaster ran in 36.46 ms of a 40 ms budget, 64% above its estimate. Every result is labelled as measured on the target or simulated on the host, and missed acceptance criteria are reported as misses.

INDEX TERMS Electric vehicles, energy management, Kalman filters, microcontrollers, processor-in-the-loop, reinforcement learning, simulation artefacts, vehicle modelling.

I. INTRODUCTION

A battery electric vehicle (EV) spends stored energy on traction, on auxiliaries such as cabin heating, and on losses that depend on how it is driven. A supervisory energy manager decides, at every control step, how much battery power the drivetrain and the heater may take. Energy management for electrified powertrains has a long record, mostly in hybrids [1], and managers trained with reinforcement learning are now a common proposal [2]. In a battery EV the manager's authority is narrow.

This paper reports an incident from a processor-in-the-loop (PIL) campaign. A Soft Actor-Critic (SAC) energy manager [3] running on an ESP32-S3 microcontroller [4] appeared to use 8.28% less energy than a rule-based manager. It was lower in 14 of 16 settings and by up to 21.6% at the highest driver aggression, and the gain grew with demand, vanished on the gentle urban and highway cycles, and exceeded what the rules' distance from the optimum allowed. Those anomalies raised the alarm. No validation gate did. The cause was a fault in the vehicle model: battery power was billed at the policy's chosen ceiling, while acceleration was computed from the torque requested before that cap. The policy learned to ask for a low ceiling and draw power above it, unbilled; a sweep with no controller in the loop reproduced the effect, with a constant 5 kW ceiling billing 8.05 Wh/km over the full US06 cycle (simulated). The gain is withdrawn here (Section IV).

The task is narrow. Speed is fixed by an inner tracking loop, and the energy manager sends three numbers: a regeneration blend, a discharge ceiling and a fraction of auxiliary load to shed. An offline dynamic-programming (DP) optimum places the rule-based manager within 0.01% to 0.16% of the best possible energy on four cycles (simulated), so a learned policy has at most that much to win. After the correction it used less energy than the rules in 1 of 16 valid settings on the target and 0.94% more on average (measured). Little room, little gain. The claim is bounded to this vehicle model, this board, one

learned algorithm and speed set by the tracking loop, and is no verdict on learned energy management in general.

The starting point of the work is the study by Tangi et al. [5]. That study predicted EV range, optimum acceleration and optimum velocity with Random Forest, Extra Trees, Linear Regression and Long Short-Term Memory (LSTM) models [6] and their ensembles. It used a synthetically generated dataset of 2,000 samples, averaged results over ten randomised partitions and served the trained models through a web application, and the full ensemble was the most accurate. The study showed that data-driven models can estimate driving parameters, and its authors named real-world data and embedded evaluation among the next steps. Three questions follow. Do such models hold on logged driving data? What happens when predictions and policies close a control loop around a vehicle? Can the chain run on an embedded controller under a real-time deadline?

The setting is PIL and not hardware-in-the-loop (HIL). The processor is real; the motor, inverter and battery are simulated. The controller runs on the microcontroller at 100 Hz, while a longitudinal vehicle model on a host computer exchanges state and commands with it every 10 ms. The vehicle model is identified and validated on logged BMW i3 trips [7], 33 of 70 of which passed the eligibility tests, and checked against the Vehicle Energy Dataset (VED) [8], while the controllers run the Urban Dynamometer Driving Schedule (UDDS), the Highway Fuel Economy Test (HWFET) and the US06 aggressive schedule [9]. Acceptance gates and pass criteria were fixed before each experiment. Misses are reported as misses. Every result is labelled measured if it ran on the microcontroller or simulated if it ran on the host only.

Where the energy was actually lost is a separate finding. Under sensor noise, the loss sits in the speed loop, below the energy manager. A two-state Kalman speed estimator [10] cut it from 9.44 to 0.24 Wh/km at ten times the nominal noise on US06 (measured). Learned reviewers of sensor faults classified better offline and controlled worse in the loop (simulated). Accuracy did not predict benefit, for the second time.

This paper makes five contributions.

- An audited case of a vehicle-model artefact that produced an 8.28% learned-policy gain, with the controller-free sweep that exposed it and three checks that would have caught it before the campaign.
- A headroom bound from dynamic programming, 0.01% to 0.16%, that explains why no honest gain was available to the energy manager.
- A corrected comparison of a rule-based manager and a SAC policy on the target, reported with paired statistics over 12 distinct conditions and the spread between repeated runs.
- A measured estimation gain in the speed loop, and the finding that offline fault-classification accuracy did not predict closed-loop benefit.
- On-device costs, timing and memory, reported next to the accuracy of each learned component.

Section II reviews related work and Section III the platform and its acceptance gates. Section IV presents the vehicle model and the artefact, Section V the corrected comparison, Section VI on-device forecasting and Section VII speed estimation and fault detection. Section VIII treats the simulated speed planner, Section IX the limits of the evidence and Section X the conclusions.

II. RELATED WORK

A. DATA-DRIVEN RANGE AND ENERGY PREDICTION

Models of EV energy use are usually fitted to logged or generated trips and scored offline. The study of Tangi et al. [5] belongs here: it compared four model families and their ensembles on a synthetic dataset and exposed them through a web application. Zhang et al. [11] used real-world driving data of 55 electric taxis in Beijing and reported a root mean squared error (RMSE) of 0.159 kWh, and Baluch et al. [12] fitted a nonlinear autoregressive model with exogenous inputs to on-road data from four trips, with a mean squared error 30% below that of an LSTM network. Gradient-boosted trees [13], Transformers [14] and physics-based loss terms [15] are further candidates. None of these studies acts on its predictions or states a run time on a vehicle-class processor.

B. LEARNED ENERGY MANAGEMENT

Reinforcement learning has been applied to energy management mainly in hybrid vehicles. A deep Q-network strategy for a power-split hybrid bus reached about 89% of the fuel economy of DP on an unseen driving condition [16], a SAC-based strategy for a hybrid tracked vehicle was compared with standard SAC and with DDPG in simulation [17], and a look-ahead strategy for a parallel hybrid combined velocity prediction with reinforcement learning and was checked on a HIL bench against DP [18]. A learned power manager for a battery EV with an ultracapacitor was compared with rules and DP [19]. The reference points are classical. DP [20] gives the optimum for a known cycle, Pontryagin's minimum principle offers an analytical route [21], and the equivalent consumption minimisation strategy (ECMS) turns the problem into an instantaneous cost [22], [23]. Rules are the usual deployable baseline.

Headroom differs between settings. In a hybrid, the split between engine and motor is a large lever. Here the manager

holds only a regeneration blend, a discharge ceiling and a shed fraction, with speed held by a tracking loop, and the DP optimum is within 0.16% of the rules. Hybrid gains do not carry over, and a null result here does not contradict them.

Evaluation practice matters as much as the method. Deep reinforcement learning results differ across seeds and implementation choices [24], and point estimates over a few runs hide uncertainty that intervals and rank statistics would show [25]. Paired differences with intervals and a rank test are reported here for that reason.

C. ECO-DRIVING AND SPEED PLANNING

When speed is free, the problem changes. Sciarretta et al. [26] cast energy-efficient driving as an optimal control problem and survey its solutions, and deep reinforcement learning has been applied to eco-driving in urban scenarios [27]. The DP planner of Section VIII belongs to the classical family. It shows where headroom reappears and is not offered as a new planner.

D. SIMULATOR ARTEFACTS AND SPECIFICATION GAMING

A learner optimises the simulator it is given, not the vehicle it stands for. Amodei et al. [28] list reward hacking among the concrete problems of learned systems, including exploits of how a reward is implemented, Skalse et al. [29] define it formally as optimisation of an imperfect proxy that lowers the true objective, and Lehman et al. [30] collected cases in which evolutionary search exposed bugs in simulations and exploited them. Sim-to-real work addresses a neighbouring gap, between simulator and physical system, by randomising simulator parameters [31] and related methods [32]. The artefact here is a plain instance of the first kind: a billing rule that did not act on motion. Section IX lists checks that would have exposed it.

E. EMBEDDED AND IN-THE-LOOP VALIDATION

Vehicle controllers are validated through a staged path of model-, software-, hardware-in-the-loop and vehicle tests [1]. HIL benches drive a real controller or real energy storage from a simulated vehicle, as in the validation of a learned power manager on an xPC Target platform [19]. PIL is an earlier step. Inference frameworks for microcontrollers [33] and joint model and library design [34] show that small devices can run networks, and integer quantisation [35] and distillation [36] shrink them. Dulac-Arnold et al. [37] list the constraints that separate laboratory reinforcement learning from deployment, among them real-time inference; what the cited works do not give, in the vehicle-energy setting, is a measured loop with a stated deadline on the target together with a rule baseline run through the same loop. This work supplies one.

F. ESTIMATION AND SENSOR-FAULT DETECTION

Kalman filtering [10] is a standard tool for state estimation, including vehicle speed; a recent example fuses inertial, steering-angle and wheel-speed sensors [38]. Model-based fault detection compares measured signals with model predictions and examines the residual [39], and learned detectors are the alternative: LSTM networks for time-series anomalies [40], a general survey [41], and the small recurrent models gated recurrent unit (GRU) [42] and FastGRNN [43]. Here, a classical estimator and a model-based detector are set against learned reviewers, each run in the vehicle loop as well as scored on labelled faults.

Table 1 places six prior works and this study along the axes that matter here. A cell is filled only where the source states the point. The sample is small and chosen for relevance, so the table positions this work and does not survey the field.

III. EXPERIMENTAL PLATFORM AND METHOD

Every result in the paper carries one of two labels. measured means the quantity was recorded while the code ran on the microcontroller; simulated means it came from the host alone. Table 2 lists every criterion fixed in advance with its outcome, and Table 3 every rule or revision made after results were seen.

A. PROCESSOR-IN-THE-LOOP ARRANGEMENT

The arrangement is processor-in-the-loop (PIL), not hardware-in-the-loop. The processor is real: an ESP32-S3 runs the

TABLE 1. Positioning of six prior studies and this work; cells marked not stated could not be confirmed from the source.

Work	Data	Closed loop	On target processor	Timing measured	Baseline
Tangi et al. [5]	synthetic	no	no	not stated	model families and ensembles
Zhang et al. [11]	logged	no	not stated	not stated	conventional method
Baluch et al. [12]	logged	no	not stated	not stated	LSTM
Wu et al. [16]	not stated	yes (simulated)	not stated	not stated	Q-learning, DP
Sun et al. [17]	not stated	yes (simulated)	not stated	not stated	SAC, DDPG
Xiong et al. [19]	not stated	yes (HIL)	real-time target (xPC)	not stated	rules, DP
This work	logged	yes (PIL)	yes	yes	rule-based

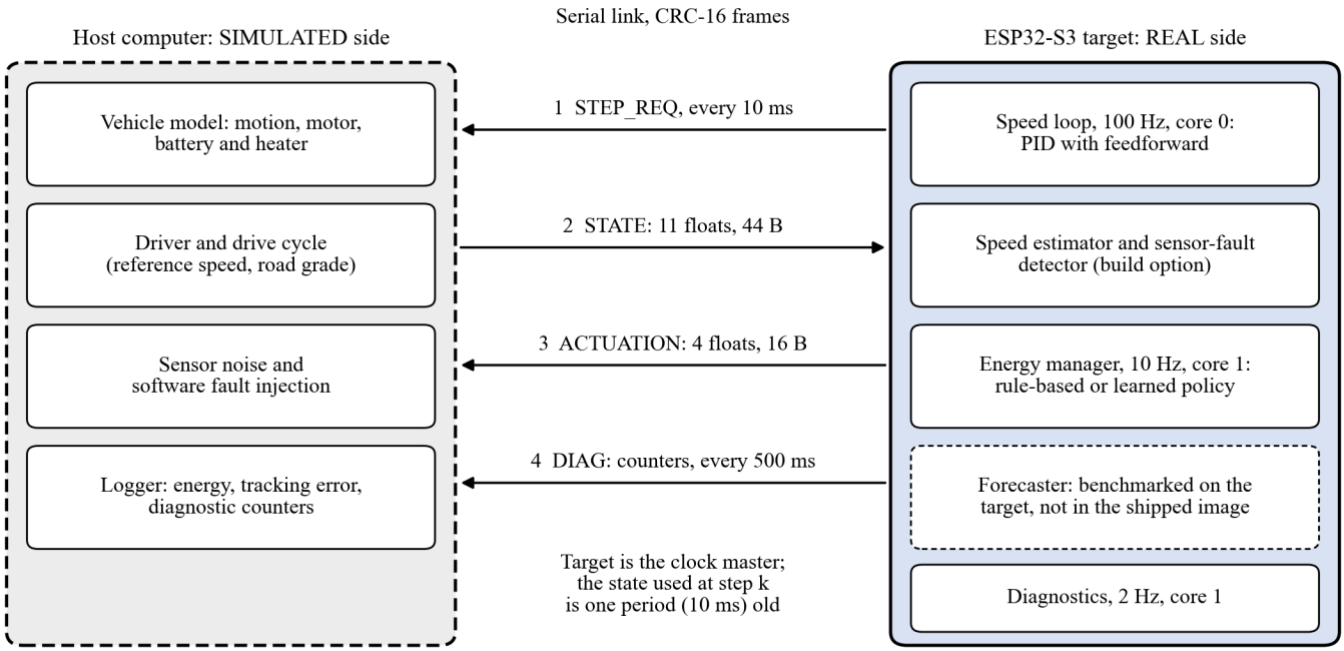


FIGURE 1. Processor-in-the-loop arrangement. The host side is simulated; the target side is real. The forecaster was benchmarked on the target but is not part of the shipped image.

control firmware, receives its inputs over a serial link and must answer before a wall-clock deadline [4]. A host laptop simulates the vehicle, motor, inverter, battery, driver and sensors. Faults are injected in software. The part is a general-purpose microcontroller, not an automotive-qualified one (Section IX).

In each 10 ms period (Fig. 1) the target, as clock master, sends a step request. The host adds seeded sensor noise to the plant state and replies with a frame of 11 floating-point values. The target returns four: torque, regeneration blend, power limit and auxiliary derate. The host steps the model and logs the result.

The firmware can block inside the slot until the reply arrives (synchronous mode) or act on the latest state already held and read replies for at most 1 ms (pipelined mode). The pipelined mode is the one reported here. The controller therefore acts on a measurement 10 ms old.

B. TARGET HARDWARE AND FIRMWARE

The target has two Xtensa LX7 cores at 240 MHz and 8 MB of PSRAM, and its FreeRTOS firmware runs four tasks. The 100 Hz speed loop has core 0 and the highest priority. It combines a PID controller with a physics feedforward and, in some builds, a speed estimator and fault detector. Core 1 hosts the energy manager and an inference task at 10 Hz and a diagnostic task at 2 Hz, so inference cannot delay control.

Which energy manager runs is a compile-time choice: the firmware builds the learned policy when its generated weight header exists and the rule-based manager otherwise. A campaign therefore needs one image per controller, and a diagnostic counter of policy steps, which cannot be non-zero in the rule-based image, confirms the choice.

The firmware was first built with the vendor framework and later moved to the Arduino core, which yields a different binary from the same source, so every gate was measured again. Throughput stayed at 29.82 million integer multiply-accumulates per second. The forecaster took 35.03 ms instead of 36.46 ms; the policy took 0.123 ms instead of 0.121 ms. The Arduino image carries radio stacks that cannot be removed. Nothing starts them, but supply current was not re-measured on this image.

C. REAL-TIME BEHAVIOUR

Gate G3 was measured over 59,650 control loops, ten minutes, against the target's own clock. On the first Arduino-core run every loop landed within 25 µs of its slot, the worst control computation took 61 µs and no loop missed its deadline (Fig. 2).

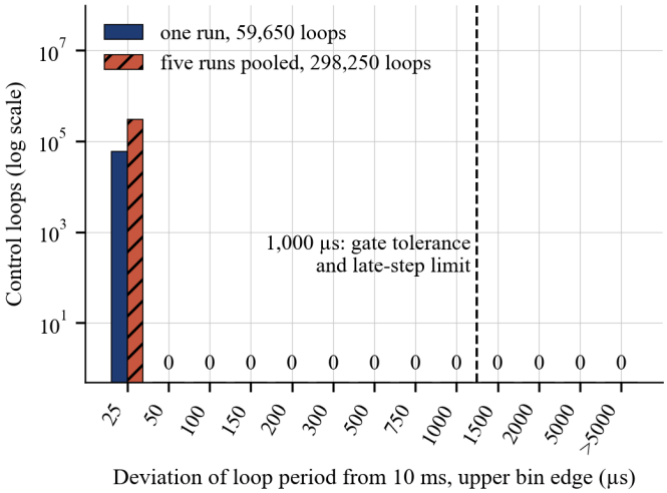


FIGURE 2. Deviation of the control period from 10 ms on the Arduino-core build (measured). Empty bins are marked 0. The five pooled runs use the shipped image with the learned policy.

The frame criterion fails. A dropped frame is a reply that misses the 1 ms window. The first run had 9. The link itself was clean. All 121,208 frames arrived, none malformed or lost, and the record attributes the misses to the host operating system answering late in bursts. Torque is ramped down only after more than five consecutive stale steps. That happened once.

Five repeats of the shipped image gave 16, 4, 6, 9 and 9 dropped frames, or 8.8 per run with a Poisson 95% interval of 6.4 to 11.8; the events cluster, so the true interval is wider. Late control steps were 0 in every run. The criterion was not loosened. Zero drops against a non-real-time host may simply not be reachable.

A stale state could bias the energy comparison of Section V. Across the 16 valid settings, the learned-minus-rules difference in dropped frames correlated with the energy difference at $r = -0.03$; no relation shows at this sample size, though a small effect is not excluded.

D. TERMS AND REPETITION

An arm is a controller under test, and a setting (or cell) is one scenario level; the campaign sweeps six factors one at a time from a US06 baseline, at three levels each, for 36 runs across two arms. The two nonzero communication-delay settings, four runs, were excluded afterwards (Table 3), leaving 32 runs in 16 valid settings. Five of the 16 are the same nominal US06 condition reached from different factors; the valid set holds 12 distinct conditions.

Energy is reported on the comparable scale: cabin heating that a controller sheds is added back at the energy it saved, and arms are ranked on the sum in Wh/km. Raw consumption would reward switching the heating off.

Seed 0, 1 or 2 is paired with the three levels of each factor and both arms receive the same draw, so within a factor the noise level is confounded with the seed, and the five nominal repeats hold at most three distinct draws. The plant and both controllers are deterministic, which means a repeat with the same seed replays the same noise draw and differs mainly in link timing, so it is not an independent repeat of anything the noise drives. The later fault campaign adds seed offsets of 100 and 200.

Anything written twice, in Python and in C, has a parity test that drives both through the same inputs; a policy network of the deployed shape, exported from an earlier seed, matches its NumPy original to 2.12e-06 with an identical weight checksum. The host is kept idle during a campaign.

E. CRITERIA FIXED IN ADVANCE, AND WHAT WAS DECIDED AFTERWARDS

A criterion is written down before the experiment it governs. A result that misses it is reported as a miss. The version history gives the dates. The five gate criteria were committed on 2026-08-30 with the design specification, before the commit that records the first gate measurement later that day. The criteria for the speed estimator and the first fault detector were committed on 2026-10-06, before any arm was run. Those for the bias-aware detector were committed on 2026-10-08 in the same commit as its code, so they preceded every learned reviewer but not the detector itself, whose step threshold was tuned on training data only.

Table 2 holds every such criterion in the paper. Five are missed. They are G2, the zero-drop part of G3, the estimator's clean-tracking bound, the first detector's false-alarm limit and the bias-aware detector's simulated detection limit. The comparison of the two energy managers had no outcome criterion; its acceptance rule, dated 2026-09-21, required only that the 36 runs be recorded, and no later record adds one, so the comparison is descriptive (Section V). Gate G4 was passed by a ridge regression before any proposed forecaster existed, which makes passing it weak evidence for the proposed model.

TABLE 2. Every criterion fixed in advance in the paper, with the date of its record and its outcome.

Id	Criterion fixed in advance	Fixed	Outcome	Label
G1	0.1 M multiply-accumulates in 20 ms or less	2026-08-30	Pass: 3.30 ms from measured throughput, margin 6.1x	measured
G2	Power RMSE within 10% of mean; final SOC within 2 points	2026-08-30	Fail on 1 of 7 trips: SOC 2.45 against 2.00	simulated
G3a	99th-percentile period jitter of 1 ms or less	2026-08-30	Pass: 25 µs	measured
G3b	Zero dropped frames in 10 min	2026-08-30	Fail: 4 to 16 per run	measured
G4	10% lower RMSE than persistence at 3 s	2026-08-30	Pass: 14.5% for the weakest fitted model	simulated
G5	40 ms or less; model plus arena 300 KB or less	2026-08-30	Timing and memory pass: 36.46 ms, 35.03 ms; 28,894 B. Converted model not yet run on target	measured
Ablations	Null result accepted in advance for physics loss and for forecast in the policy state	2026-08-30	Both null; forecast ablation on the corrected model +0.262 Wh/km, interval −0.698 to +0.982	simulated
Speed estimator	Clean energy within 0.2% of the baseline loop	2026-10-06	Pass	measured
Speed estimator	Clean tracking within 0.005 m/s of the baseline loop	2026-10-06	Fail: 0.0193 against 0.0104 m/s on UDDS	measured
Speed estimator	At least half of the 10x-noise penalty recovered	2026-10-06	Pass: 9.44 to 0.24 Wh/km	measured
Speed estimator	Tracking below 0.1 m/s in every cell	2026-10-06	Pass	measured
First detector	No false alarm on a healthy sensor	2026-10-06	Fail: 1 false alarm on the target	measured
Bias-aware detector	Every named fault flagged within 0.5 s; under 1 false alarm per clean hour	2026-10-08	Not met: 551 and 533 of 560 caught	simulated
Bias-aware detector	Same limits on the target	2026-10-08	Met on cases run: 120 of 120 within 0.32 s; 0 alarms in 1 clean hour	measured

Table 3 records each rule or revision made after results were seen. The re-run rule and the delay exclusion are judgement calls. The origin of the 16-frame threshold is not recorded; it equals the largest count among the five G3 repeats, but the record does not say it came from them.

TABLE 3. Rules and revisions made after results were seen.

Change	Recorded	Prompt	Effect
G2 split into three parts	2026-08-30	Pooled form failed all 7 trips	Both thresholds kept, a torque bound added; gate fails 1 of 7
Delay settings excluded	2026-09-21	Neither arm followed the cycle (25.7 to 43.4 m/s)	Rule: tracking error 1 m/s or more; 4 of 36 runs dropped
Re-run above 16 dropped frames	2026-10-07	Four rule cells run beside other jobs (21 to 54)	Looser than G3; 8 cells later set aside
Plant fix and added checks	2026-10-06	Anomalies in campaign tables, not a gate	8.28% gain withdrawn; limit-invariance sweep and optimum check added
Second fault detector	2026-10-08	First detector assumed an acceleration channel that logged trips did not match	Offset state added; criteria recorded with it
Fused backup speed	2026-10-09	Checker with rule monitor failed on a heavier car	18 candidates on one training cycle; rule fixed before runs

IV. VEHICLE MODEL AND THE POWER-LIMIT ARTEFACT

Every controller in this study acts on a simulated vehicle, so a fault in that vehicle enters every comparison. One such fault produced an 8.28% mean energy saving for the learned policy that did not exist. This section describes the model, what its validation did not test, and the fault, its discovery and the checks it implies.

A. DATA

The model was identified and validated on a public archive of 70 trips of a BMW i3 [7], logged every 0.1 s. Eligibility tests were fixed before any fit: state of charge (SOC) present, no charging, and two heater-power channels present and in agreement. Thirty-three trips passed, all in winter, so the validated envelope has a mean ambient temperature from −3.3 to 10.2 °C. The 33 were split by trip into 19 for identification, 7 for validation and 7 sealed for a final test that has not been opened.

The Vehicle Energy Dataset (VED) [8] served for a transfer check. The campaigns use three dynamometer schedules [9]: the Urban Dynamometer Driving Schedule (UDDS, 1369 s, 11.99 km), the Highway Fuel Economy Test (HWFET, 765 s, 16.51 km) and the aggressive US06 schedule (600 s, 12.89 km).

B. LONGITUDINAL MODEL

The force balance (1) sets the acceleration, with mass m, rotational mass factor e, speed v, wheel force F_t, rolling and drag coefficients c_{rr} and c_d, frontal area A, air density ρ and grade θ.

$$m(1+e)\dot{v} = F_t - mgc_{rr}\cos\theta - \tfrac{1}{2}\rho c_d A v^2 - mg\sin\theta \tag{1}$$

Torque T becomes wheel force through the gear ratio, wheel radius and drivetrain efficiency. Electrical power uses motor speed ω and an efficiency η from a map identified on training trips only.

$$P_e = T\omega/\eta(T,\omega)\ \ (T\omega>0), \qquad P_e = T\omega\,\eta(T,\omega)\ \ (T\omega<0) \tag{2}$$

The battery is a first-order equivalent circuit with terminal voltage V_t, open-circuit voltage V_{oc} at SOC s, current i, resistance R_0 and an R_1C_1 branch; its open-circuit curve, branch values and current limits are assumptions.

$$V_t = V_{oc}(s) - iR_0 - v_1, \qquad \dot{s} = -\frac{i}{3600\,C_b}, \qquad \dot{v}_1 = \frac{i}{C_1} - \frac{v_1}{R_1C_1} \tag{3}$$

The auxiliary load is 543 W at rest (measured) plus 127.3 W per degree below a 21 °C setpoint (fitted); the cooling slope is unidentified, because every eligible trip is colder than the setpoint. Regenerative braking returns a fraction of the braking power, limited by an assumed 50 kW ceiling and the pack's charge current. Table 4 lists the main parameters.

TABLE 4. Main parameters of the i3 model, with the origin of each value.

Parameter	Value	Origin
Mass m	1270 kg	published kerb mass plus driver
Frontal area A	2.38 m²	from published drag area and an assumed drag coefficient
Wheel radius, gear ratio	0.3498 m, 9.665	from tyre size; published
Motor limits	250 Nm, 125 kW	published
Drivetrain efficiency η_d	0.95	assumed
Rolling coefficient c_{rr}	0.01947	identified, training trips
Drag coefficient c_d	0.2202	identified, training trips
Rotational factor e	0.1706	identified, training trips
Resistance R_0	0.010 ohm	identified, at lower bound
Capacity C_b	61.7 Ah	Coulomb counting of logged data

C. VALIDATION AGAINST LOGGED TRIPS

Gate G2 was written before the fit. It replays a held-out trip's logged speed and passes only if every validation trip meets the bounds. The original single criterion pooled three models and failed all seven trips without saying which was at fault, so it was split into three parts with both original thresholds kept. Part A: torque inverted from the speed profile within 50% of the logged-torque standard deviation, the only new number. Part B: battery power, driven by logged torque, within 10% of mean absolute power. Part C: terminal SOC within 2 percentage points (pp).

The gate fails on Part C. Parts A and B pass on 7 of 7 trips (worst case 48.6% and 8.3%), and Part C on 6 of 7, with a median of 0.19 pp. Trip B08 misses by 2.45 pp against 2.00 pp (Fig. 3); the bound was kept and the miss recorded.

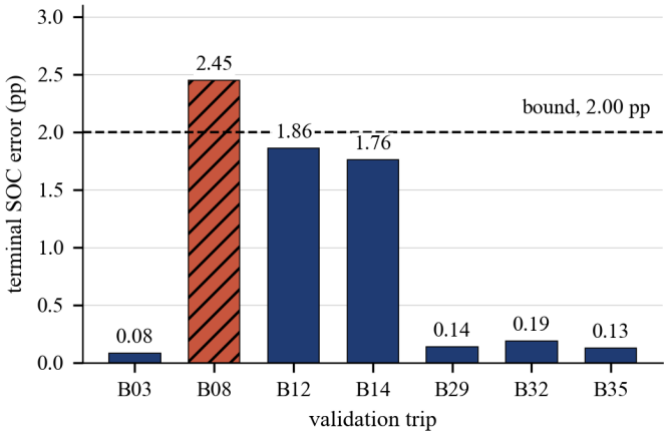


FIGURE 3. Terminal SOC error on the seven validation trips against the 2.00 pp bound of gate G2. Trip B08 (hatched) exceeds it. Replay on the host (simulated against logged trips).

G2 checks the model along logged trajectories: logged torque goes in, and battery power and SOC come out. The controllers here send the model a battery power ceiling, and a replay of logged torque never asks the model to enforce one. The path that carried the fault below lay outside everything G2 exercised, so the gate could not have failed on it.

D. TRANSFER TO ANOTHER VEHICLE

The model was rebuilt around a 2013 Nissan Leaf from the battery-electric subset of VED and run on 89 validation trips. Arm A carried every i3 parameter across, arm B replaced the chassis coefficients with the Leaf's coastdown values, and arm C refitted the chassis. Median trip energy error was 16.9%, 14.4% and 13.4% (simulated against logged trips).

E. THE EARLIER CAMPAIGN AND THE APPARENT GAIN

The first processor-in-the-loop (PIL) campaign ran both energy managers on the ESP32-S3 target [4] against this model, over six factors at three levels each: 36 runs, 32 of them valid (Section V). The Soft Actor-Critic (SAC) policy [3] used less energy than the rule-based manager in 14 of 16 valid settings, by 8.28% on average and by up to 21.6% at driver aggression 1.5, on the comparable energy scale of Section III (measured). Fig. 4 shows each setting.

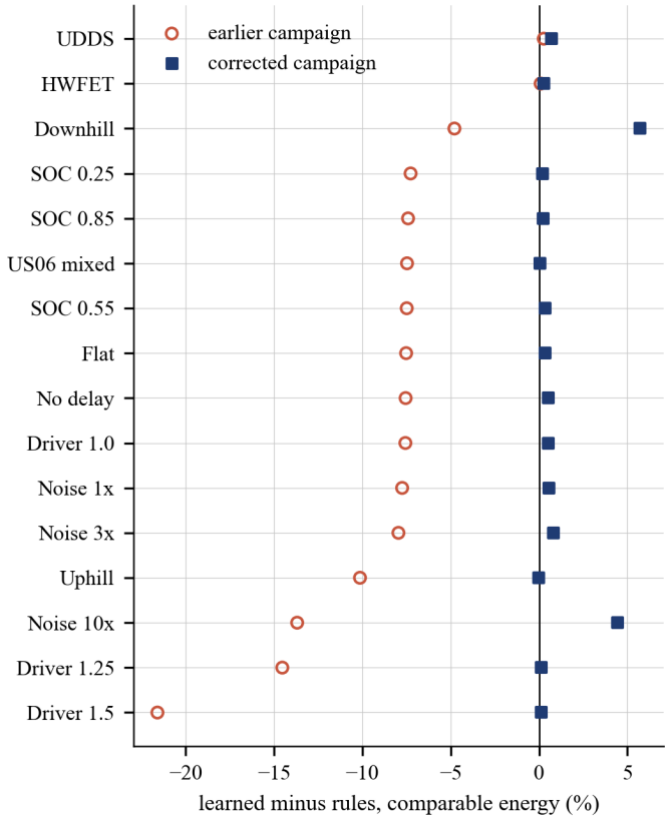


FIGURE 4. Learned minus rule-based energy per setting on the comparable scale, in the earlier campaign on the faulty model and in the corrected campaign. Negative means the learned policy used less. measured, one run per cell; five of the 16 settings repeat the nominal US06 condition.

The gain rose with demand on three axes: 7.57%, 14.54% and 21.61% across driver aggression 1.0, 1.25 and 1.5, and 4.79%, 7.53% and 10.14% across downhill, flat and uphill. On the two gentlest cycles it was absent, with the learned policy using 0.27% (UDDS) and 0.09% (HWFET) more. A repeat campaign the next day, on firmware that also logged the discharge ceiling, reproduced it at 14 of 16 settings and 8.29% (measured). The ceiling averaged 72.0 kW for the learned policy and 123.4 kW for the rules over the valid settings. A repeat shows that a result is reproducible on a model, not that the model bills fairly.

F. THE MECHANISM

Each energy manager sends the model a ceiling P_{lim} on battery discharge power. The model applied it to the bill and not to the motion.

$$P_b = \min(P_m + P_a,\ P_{lim}), \qquad a = f(T_{req}) \tag{4}$$

Here P_m is the machine power, P_a the auxiliary load and T_{req} the torque requested before any cap. The battery was charged P_b while acceleration came from T_{req}, so billed and delivered power parted wherever demand exceeded the ceiling. A

controller that lowered the ceiling was charged less and still received the motion it asked for.

The cause was isolated with no controller in the loop. A constant ceiling was held over US06 on the model before the fix, read from the repository history (Fig. 5, simulated on the host). A 5 kW ceiling drove all 12.888 km at 8.05 Wh/km, with the same 0.0013 m/s tracking error as an open ceiling at 190.39 Wh/km. A constant 40 kW ceiling billed 174.85 Wh/km. The deployed learned policy, simulated on that model, scored 176.71 Wh/km against 176.53 measured on the target.

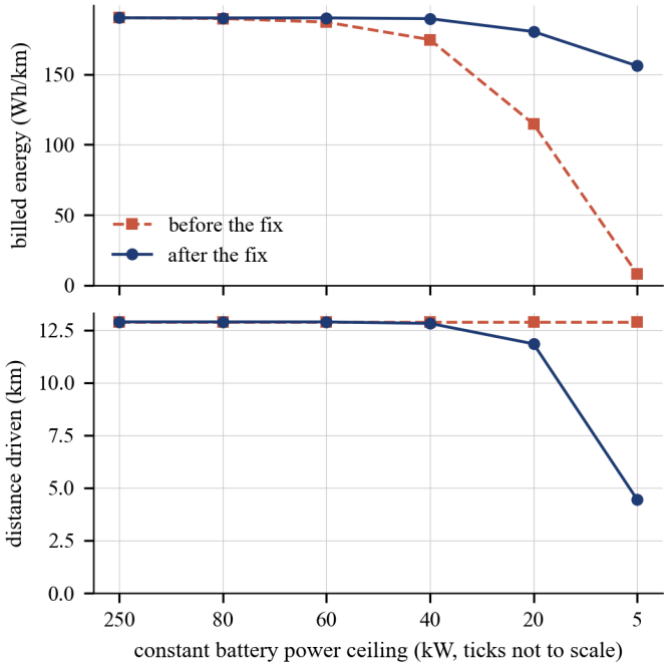


FIGURE 5. A constant battery power ceiling held over US06, before and after the fix, with no controller in the loop. Before, a lower ceiling cut the billed energy and left the distance unchanged; after, it also cuts the distance. simulated on the host.

G. HOW IT WAS FOUND

The gates did not find it. G2 never exercised the ceiling path. The campaign tables carried three cues. The learned policy finished 7% below the offline dynamic-programming (DP) optimum of 190.39 Wh/km at equal tracking, which should not be possible; its advantage grew with demand; and it vanished on UDDS and HWFET, where demand stays under the 40 kW floor of the ceiling range. The fault was found 15 days after the first campaign.

Two explanations were recorded on the way and both were wrong. The first campaign's note read the cues as peak-current shaving and marked the reading as inferred, because the ceiling had not been logged. The repeat campaign showed that both controllers used the same 40 to 125 kW range, which corrected a belief that only the learned policy limited power, but it did not reveal the fault. The controller-free sweep did.

The audit began with a result that favoured the learned policy. A fault that made it look worse would not have drawn the same scrutiny, and no symmetric audit was run.

H. THE FIX AND ITS CONSEQUENCES

The fix gives the motor the power the pack delivered less the auxiliary load, reduces torque to match, and computes acceleration from the reduced torque. Four unit tests and one cycle-level test were added. Two limits that never bind give bit-identical outputs, and a starving ceiling must cost distance and tracking, not only the bill.

On the corrected model a 5 kW ceiling gives 156.16 Wh/km over 4.453 km with 16.15 m/s tracking error, and a 40 kW ceiling gives 189.80 Wh/km against 174.85 before (simulated). The rule-based manager and the DP optimum did not change, since their ceiling never bound. The learned policy on US06 moved from 176.71 to 190.71 Wh/km, against 190.47 for the rules, with tracking error 0.48 m/s against 0.0013 m/s. On the target, across 16 valid settings, the corrected rule arm differed from the repeat campaign by +0.07% on average (95% interval −0.03 to +0.17%), 0.46% at worst, against 0.57% at worst between two campaigns that differed only in the day (measured). The corrected comparison is in Section V.

I. CHECKS THE CASE IMPLIES

Table 5 lists three checks that fail on the faulty model. All three were written after the fault and applied to the pre-fix model; none was a gate fixed in advance.

TABLE 5. Audit checks and their outcome on the faulty model. simulated on the host.

Check	Test	Faulty model
Billing against motion	Constant ceiling over a cycle; energy and distance must fall together	Fails: 8.05 Wh/km, distance unchanged
Learner against optimum	Learned energy at equal tracking must not beat the DP optimum on the same model	Fails: 176.71 against 190.39 Wh/km
Limits act on motion	A binding limit must reduce delivered traction; a non-binding one must change nothing	Fails: tracking stayed at 0.0013 m/s

The second check applies to any learned controller: a learner that beats an optimum computed on the same model has found either a larger search space or a hole in the model.

J. LIMITS OF PLANT FIDELITY

The three identified chassis coefficients do not measure what they are named after. The fitted rolling coefficient of 0.01947 is 2.2 times the 0.00878 of a published coastdown test of a 2013 Nissan Leaf [44]. That is unlikely to be tyre rolling resistance alone; the value probably absorbs drivetrain and brake drag, efficiency-map error and unmodelled load, and the same holds for c_d and the rotational factor of 0.1706. The 50 kW regeneration ceiling and the 0.95 drivetrain efficiency are assumptions, and R_0 sits at its lower bound. The envelope is winter only, the campaigns ran at 12 °C, G2 fails on 1 of 7 trips, and on transfer the median energy error is 13.4% to 16.9%.

These limits threaten absolute figures first: Wh/km values in this paper belong to this model, not to a car. A paired comparison on one model is protected against errors that act on both arms alike, such as a high rolling coefficient. It is not protected against an error that meets one arm's action space, which is what the artefact was. The regeneration ceiling and the efficiency assumption were not tested against the learned policy in this way, and no sensitivity study of the verdict to these parameters was run. The headroom bound of Section V comes from the same model and inherits the same limits.

V. ENERGY MANAGEMENT AFTER THE CORRECTION

This section compares a hand-written energy manager with a learned one on the corrected vehicle model of Section IV. Both run on the microcontroller. The question is narrow. With the speed profile fixed by a tracking loop and only three actuation numbers to choose, can a learned policy use less energy than the rules, on the corrected vehicle model? It cannot.

A. CONTROL STRUCTURE AND THE RULE-BASED MANAGER

Two loops share the vehicle. An inner speed loop runs at 100 Hz on the microcontroller and turns the speed reference into motor torque. A slower supervisory energy manager runs at 10 Hz and sends three numbers: the fraction of braking energy routed to the machine (regeneration blend), a ceiling on battery discharge power, and the fraction of the auxiliary load, mainly cabin heating, to shed. The energy manager cannot change the speed reference, so no controller can save energy by driving slower.

The rule-based energy manager is a pure function of state of charge (SOC) and speed, with no memory. Regeneration is full below SOC 0.85, tapers linearly to zero at 0.94, and fades out below 2 m/s so that stops stay predictable. The discharge ceiling is 125 kW above SOC 0.20 and falls linearly to 40 kW at SOC 0.07. Auxiliary shedding stays at zero above SOC 0.15 and rises to a cap of 30% as the pack empties. Two of these constants copy plant limits (0.94 is the battery's maximum SOC and 0.07 its minimum); the others were set by engineering judgement and not tuned against any test cycle. Table 6 lists them next to the learned policy's settings.

B. HEADROOM UNDER A FIXED SPEED PROFILE

An offline dynamic-programming (DP) optimum [20] was computed on the WLTC class 3b cycle and the three dynamometer schedules UDDS, HWFET and US06 [9], at a starting SOC of 0.80 and 12 degrees C. The rule-based manager sits within 0.01% to 0.16% of it: 153.90 against 153.89 Wh/km on the highway cycle (HWFET), 167.93 against 167.67 on the urban cycle (UDDS), 190.47 against 190.39 on the aggressive cycle (US06) and 175.99 against 175.94 on WLTC class 3b (simulated). The optimum is a constant instruction: recover as much braking energy as the pack accepts. The DP held the discharge ceiling at its maximum, and the HWFET rule value is unchanged on the corrected model.

The bound follows from the structure. Acceleration depends on torque and grade alone, so regeneration blend and shedding cannot alter the speed trajectory, and a ceiling can only remove power the speed loop needs, which the tracking constraint forbids. Shedding is priced at the energy it saves, which makes it neutral. A learned policy in this task therefore has at most 0.16% to gain, and the experiment cannot tell a good learner from a poor one by energy alone. A gain needs a task with headroom, and Section VIII shows where one exists.

C. THE LEARNED POLICY

The learned policy is trained with Soft Actor-Critic (SAC) [3]. It observes eight scaled quantities: SOC, speed, acceleration, battery temperature, road grade, tracking error, and the mean and maximum of the next 3 s of reference-implied traction power. It outputs the same three numbers as the rules, mapped affinely from $[-1, 1]$ to the physical ranges. The reward is the negative sum of battery energy, shed auxiliary energy at the price it saves, a tracking-error term and penalties for excess current and low SOC; Table 6 gives the weights. The deployed actor is 8-64-64-3 with 4,931 parameters. Training used the WLTC3B and UDDS cycles with the HWFET cycle for validation, five seeds of 200,000 steps each.

Only one reward design and one hyperparameter set were run. There was no sweep. The tracking deficit reported below may reflect the tracking weight. This was not tested.

TABLE 6. Settings of the learned policy and the rule-based manager. Library values are defaults of the Stable-Baselines3 SAC class; the others are set in the training script.

Item	Value	Origin
Actor and critic hidden layers	64, 64	script; 4,931 actor parameters
Learning rate	0.0003	library default
Batch size, replay buffer	256, 1,000,000	library default
Discount, soft-update rate	0.99, 0.005	library default
Entropy coefficient	automatic	library default
Steps before learning starts	1,000	script
Training steps, seeds	200,000, 5	script
Episode length	300 outer steps (30 s)	script default
Reward: energy, shed heat	1.0 per Wh each	script
Reward: tracking	5.0 per m^2/s^2	script
Reward: current, low SOC	2.0 per A s over limit; 50 per unit SOC below floor plus 0.02	script
Rule thresholds	see text	judgement; two equal plant limits

The comparable energy scale charges shed cabin heat back at exactly the energy it saved, so a policy that switches the heater off cannot look efficient. All ranking uses it.

After the model correction, five seeds per arm were retrained and scored on HWFET (simulated). The arm without forecast channels reaches 154.86 ± 0.51 Wh/km (seed standard deviation) against 153.90 for the rules, and the arm with them reaches 155.12 ± 0.82. The no-forecast seed with the lowest validation energy, seed 3 at 154.21 Wh/km, was exported to the target. It costs 0.121 ms per decision on average and 19.3 KB on the ESP32-S3 [4], and its outputs match the host implementation to 2e-06 against a tolerance of 2e-05 (measured, on another seed of the same network). The worst policy computation in the campaign loop took 0.84 ms.

D. FORECAST ABLATION

The forecast ablation removes two observation channels, the mean and maximum of the next 3 s of reference-implied traction power, which are computed from the reference speed. It is not the Transformer forecaster of Section VI, which never ran inside the energy-management loop. Seeds are paired. On the original vehicle model the difference (with minus without) is +0.240 Wh/km with a 95% bootstrap interval of −0.669 to +1.228; after retraining on the corrected model it is +0.262 with an interval of −0.698 to +0.982 (both simulated, five paired seeds). Both intervals contain zero. No equivalence margin was set, so the result is the absence of a detected effect and not proof of none. Under a fixed speed profile the best action does not depend on what happens next, which agrees with the DP result. The deployed policy was trained without them.

E. CORRECTED CAMPAIGN ON THE TARGET

The campaign closes the loop in the processor-in-the-loop (PIL) arrangement: the control software runs on the microcontroller at 100 Hz, and the host simulates the vehicle, motor, inverter and battery. An arm is one controller under test; a setting is one combination of conditions. Six factors were each swept across three levels, one at a time from a US06 baseline at 12 degrees C: driving cycle (UDDS, HWFET, US06), road grade (+0.03, 0 and −0.03 rad), starting SOC (0.85, 0.55, 0.25), sensor noise (1x, 3x and 10x a stated floor), communication delay (none, 10 ms, 30 ms) and driver aggression (a time scaling of the reference by 1.0, 1.25 and 1.5). Each setting ran once per arm: 36 runs, 32 of them in the valid set, and 2,282,832 control steps, none late against the loop's own deadline (measured).

The two delay settings above zero are excluded because the host cannot answer within the delay and neither controller follows the cycle (tracking error 25.7 to 43.4 m/s); the exclusion rule, a tracking error of 1 m/s or more, was fixed after those runs were seen, and the largest valid error is 0.504 m/s. The remaining 16 settings are not 16 distinct conditions. Five of them, labelled R in Table 7, are the same nominal US06 condition (mixed driving, no delay, flat road, 1x noise, driver 1.0), reached through different axes; the SOC 0.55 setting starts at a different charge and is not one. The valid set therefore holds 12 distinct conditions. Three of the five repeats used noise seed 0, so they differ mainly in link timing and run order, which is why their spread is a floor on run-to-run variation and not a full estimate of it.

The repeats give the run-to-run spread directly. The rule arm used 190.73 to 191.41 Wh/km across them, a range of 0.36%, and the learned arm 191.43 to 191.98, 0.29%. The paired difference (learned minus rules) is +0.40% on average with a standard deviation of 0.22 and a 95% t-interval of 0.13

TABLE 7. Corrected PIL campaign, 16 valid settings, one run per setting per controller, comparable energy scale. R marks the five repeats of the nominal US06 condition; Mixed is that condition reached through the driving-condition axis. measured.

Setting	Cycle	Rules (Wh/km)	Learned (Wh/km)	Difference (%)	Rules RMSE (m/s)	Learned RMSE (m/s)
Urban	UDDS	168.13	169.30	+0.70	0.013	0.016
Highway	HWFET	153.96	154.40	+0.29	0.004	0.004
Mixed (R)	US06	191.41	191.48	+0.03	0.044	0.226
No delay (R)	US06	190.73	191.76	+0.54	0.014	0.230
Uphill 0.03 rad	US06	305.26	305.21	−0.02	0.039	0.237
Flat (R)	US06	190.75	191.43	+0.36	0.015	0.229
Downhill 0.03 rad	US06	79.30	83.83	+5.71	0.010	0.193
SOC 0.85	US06	190.93	191.37	+0.23	0.026	0.233
SOC 0.55	US06	191.00	191.63	+0.33	0.027	0.195
SOC 0.25	US06	190.80	191.20	+0.21	0.021	0.168
Noise 1x (R)	US06	190.75	191.81	+0.55	0.021	0.230
Noise 3x	US06	191.33	192.90	+0.82	0.014	0.218
Noise 10x	US06	201.55	210.53	+4.45	0.055	0.192
Driver 1.0 (R)	US06	190.99	191.98	+0.52	0.028	0.228
Driver 1.25	US06	197.16	197.38	+0.11	0.045	0.331
Driver 1.5	US06	204.46	204.75	+0.14	0.098	0.504

to 0.67 (n = 5). The difference in dropped frames between the two arms is unrelated to the energy difference (r = −0.03 over the 16 settings).

Table 7 and Fig. 6 give the per-setting result. The learned policy uses less energy in 1 of the 16 settings, uphill at −0.02%, which is 0.05 percentage points below the lowest nominal repeat and smaller in size than the 0.36% spread of the rule arm. Averaged over the 16 settings it uses 0.94% more, with a median of +0.35%. Counting each distinct condition once, with the five repeats merged into their mean, the learned policy is higher in 11 of 12. The mean difference is +1.11% (bootstrap 95% interval 0.27 to 2.27) and the median +0.31% (0.17 to 0.76). An exact Wilcoxon signed-rank test gives p = 0.001 and a sign test p = 0.006, both two-sided and over the 12 conditions. Two settings carry the mean. These are downhill (+5.71%) and 10x sensor noise (+4.45%).

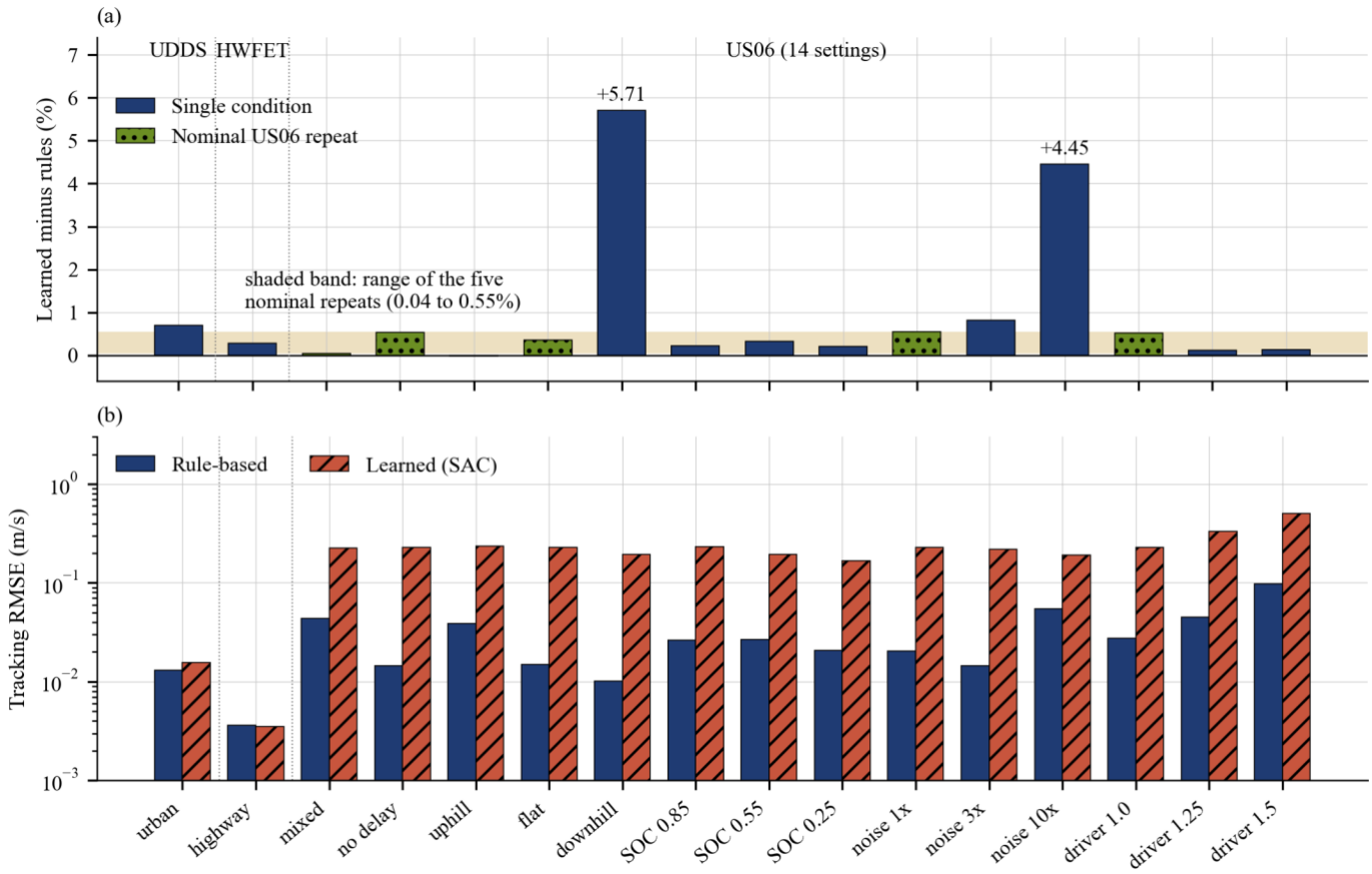


FIGURE 6. Corrected campaign, 16 valid settings: (a) paired energy difference of the learned policy relative to the rules, positive meaning more energy, with the range of the five nominal repeats as a shaded band and the repeats hatched; (b) tracking RMSE of both controllers on a log axis. measured, one run per setting.

On tracking, the learned policy has a root-mean-square error (RMSE) of 0.17 to 0.24 m/s on most US06-based settings, 0.33 m/s at driver aggression 1.25 and 0.504 m/s at 1.5, against 0.010 to 0.098 m/s for the rules. On UDDS and HWFET the learned policy stays at 0.016 and 0.004 m/s. US06 was held out of training, but it was also the cycle of the earlier campaign that led to the model fix, so it was not unseen by the authors, and the deficit appears on the cycles built on it.

The verdict covers this task only: one vehicle model at one ambient temperature, one board, one learned algorithm with one reward design, and speed fixed by the tracking loop.

F. THE EARLIER CAMPAIGN

An earlier campaign on the uncorrected model showed the opposite sign: the learned policy used less energy in 14 of the same 16 settings, by 8.28% on average. That gain came from the vehicle model, not from the policy. Section IV gives the mechanism, the controller-free sweep that exposed it and the checks it implies. After the fix and retraining it disappears.

G. BEHAVIOUR UNDER LINK OUTAGE

When the host link drops, the firmware reacts in two ways. If the 100 ms energy-manager setpoint is more than 300 ms old, the rule-based action runs inside the speed loop. If no vehicle state has arrived for more than five control periods (50 ms), the torque command is ramped to zero at 10 N m per step and the integrator is frozen. The fallback ablation compiles these two behaviours out; the firmware then holds the last command.

The sweep used outage lengths of 0.2, 0.5, 1, 2, 3, 4, 5 and 10 s, eight paired onsets on US06 per length, with the retrained policy in the loop and the corrected model (measured). Fig. 7 shows the worst peak speed error. Removing the handling gives the smaller worst error from 0.2 s to 4 s (0.18 against 0.23 m/s at 0.2 s, 3.73 against 6.67 m/s at 4 s). At 5 s the order reverses, 7.40 m/s with the handling against 7.97 without, so the crossover lies between 4 and 5 s; no point inside that interval was measured. At 10 s the worst error is 13.3 m/s with the handling and 18.2 m/s without. The median is less regular: the handling has the lower median at 4

s (1.21 against 1.55 m/s) and at 10 s (3.72 against 5.33 m/s), and the higher one at 5 s (1.64 against 1.44 m/s).

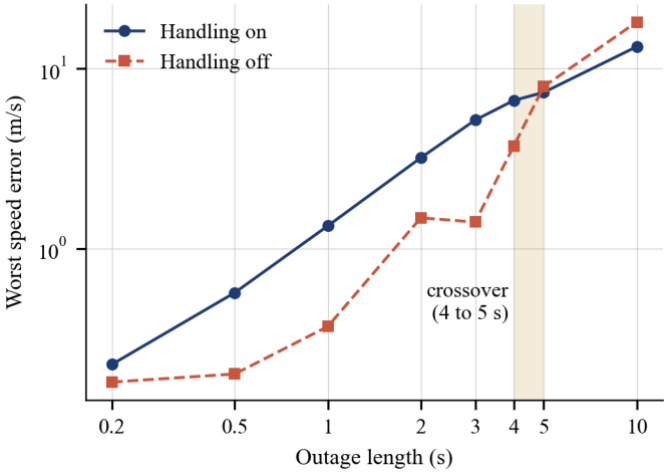


FIGURE 7. Worst peak speed error against outage length, with the fallback handling on and off, on the corrected model. measured, eight onsets per length; lines join measured points only.

Holding a stale command works for a second or two and fails once the reference has moved on; the handling gives up accuracy early and loses less late. This is a description of behaviour under a link fault, not a functional-safety claim.

VI. ON-DEVICE FORECASTING: ACCURACY AGAINST COST

This section is a supporting result. A forecast of battery power for the next few seconds has to run on the same microcontroller as the control loop, so the question here is how accurate such a forecast can be and what each model costs on the ESP32-S3. The energy manager of Section V did not use any of these forecasters. Accuracy was computed on the host from validation trips (simulated); timings marked measured were taken on the target. The test trips were not opened.

A. TASK AND SAFEGUARDS

The task is to predict battery power at each of the next 30 samples, a horizon of 3.0 s at 10 Hz, from the preceding 32 samples (3.2 s) of eight signals: speed, acceleration, state of charge (SOC), battery current, battery voltage, ambient temperature, road grade and auxiliary power. Splits are by trip: 55,125 training windows from 19 trips and 29,252 validation windows from 7 trips.

Three safeguards were fixed before any model was fitted. Battery power is voltage times current and both are inputs, so inputs end at sample 32 and targets begin at sample 33, which an automated test asserts. A ridge regression with 257 parameters scored an R^2 of 0.4018 on real targets and -0.0044 on shuffled ones. Persistence holds the last value. It gives 13.219 kW of root-mean-square error (RMSE) at 3 s, and gate G4 requires 10% below that, or 11.897 kW.

B. ACCURACY AGAINST COST

The ladder runs, in order of cost, from persistence to linear regression, gradient-boosted trees (XGBoost) [13], a one-layer long short-term memory network (LSTM) [6] and a two-layer Transformer encoder [14]. Table 8 and Fig. 8 give the result. Each neural model is scored at its best validation epoch, so its error is slightly optimistic.

Every fitted model passes G4, by at least 14.5%. That settles little, because linear regression had already cleared the bar before the larger models existed, so the pass shows that a forecast is possible and says nothing in favour of a Transformer. Linear regression gives 87% of the Transformer's gain for 1.2% of its estimated compute. XGBoost is about twelve times over the 300 KB memory budget and stays as an offline reference. Seed spread within a model, 0.038 to 0.174 kW, is as large as the gaps between the top three.

TABLE 8. Forecast error at 3 s on validation trips (simulated) and cost on the ESP32-S3. Latency is measured where stated and otherwise estimated from operation counts.

Model	Error at 3 s (kW)	Gain over persistence (%)	Size (KB)	Latency (ms)	Share of 40 ms budget (%)
Persistence	13.219	0	0	0	0
Linear regression	11.308	14.5	7.5	0.26 (estimated)	0.6 (estimated)
XGBoost	11.290	14.6	about 3,658	not applicable	not applicable
LSTM	11.120	15.9	20.4	35.07 (measured)	88
Transformer	11.029	16.6	17.9	36.46 (measured)	91

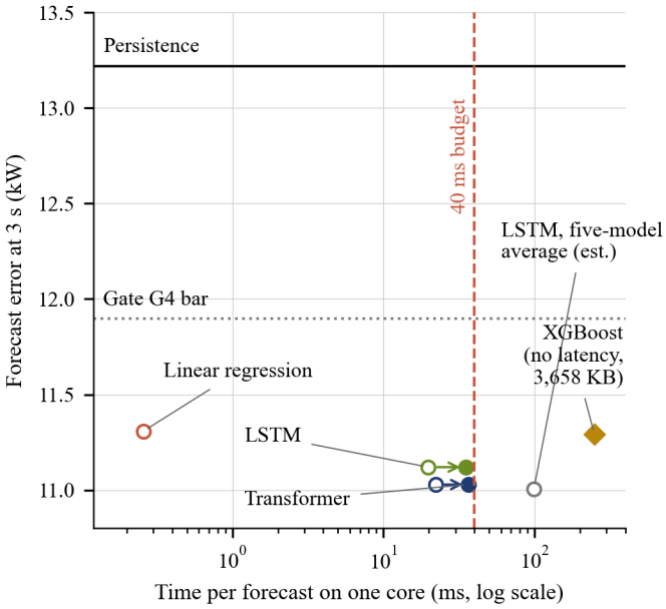


FIGURE 8. Forecast error at 3 s against time per forecast. Open markers are estimates from operation counts; filled markers are measured on the ESP32-S3 (vendor-framework build). XGBoost has no latency figure and sits at the right edge.

C. COST ON THE TARGET

The first Transformer cost, 22.28 ms, was the multiply-accumulate count divided by the throughput measured on the target. The same network in 8-bit C then took 36.46 ms per forward pass on the vendor-framework build, 91% of the 40 ms budget of gate G5, and 35.03 ms on the Arduino core (measured). The measurement was 64% above the estimate. Softmax, which needs 1,024 exponentials per head in each of four heads and two layers, and layer normalisation together account for 7.89 ms of the 14.18 ms gap, and the rest is not explained in the record. The LSTM has no softmax and was further off, 19.84 ms estimated against 35.07 ms measured, so the optimism lies in the method. Not the architecture. Operation counts can rank models. They should not set budgets. The timed code uses fixed weights, so it times the graph and says nothing about what a trained model would predict.

Rounding weights and activations to 8 bits [35] raised the error of the final-epoch model from 11.051 kW to 11.156 kW, a loss of 0.95%, and shrank the stored model 3.6 times, from 71.7 KB to 20.2 KB (simulated). The timed graph needs 28,894 bytes, 9.4% of the memory budget. Gate G5 is not closed in full. The converted model has not been run through an embedded runtime on the target, so the 8-bit accuracy is a host figure only.

Two further studies, both run on the host, added nothing to accuracy. A physics-informed loss [15] left the error unchanged, 11.057 kW against 11.052 kW, though it cut the share of steps where power disagrees with voltage times current by more than 500 W from 24.5% to 2.5%; consistency improved and accuracy did not. Distillation from a larger teacher [36] did not help. The students were worse by 0.20% and 0.89%, and the sign changed between seeds.

VII. SPEED ESTIMATION AND SENSOR-FAULT DETECTION

The one measured gain of the study sits in the speed loop, a level below the energy manager of Section V. Results are measured when they ran on the ESP32-S3 in the processor-in-the-loop (PIL) arrangement, and simulated when they ran on the host only. Faults were injected in software, and no physical accelerometer was used. The detector and fallback are evaluated as monitoring functions only; no ISO 26262 or safety-of-the-intended-functionality claim is made.

A. WHERE THE NOISE LOSS SITS

Sensor noise raises the energy used under the rule-based manager. On US06 the energy, on the comparable scale of Section III, rose from 190.98 Wh/km at the noise floor to 200.42 Wh/km at ten times the floor (measured). The speed loop turns the reading into a torque command at 100 Hz, so noise becomes alternating drive and brake commands. Either the manager decides worse on a noisy state, or the loop wastes energy on the reversals.

Simulation supports the second reading: with the manager fixed, replacing only the speed estimate cut 200.3 to 190.6 Wh/km at ten times the noise and torque reversals from 1,342 to 83 per km (simulated, five seeds).

B. TWO-STATE KALMAN SPEED ESTIMATOR

The estimator is a Kalman filter [10] over the speed v and an unknown acceleration d, which absorbs what the vehicle model gets wrong, such as extra mass or headwind. The prediction advances the speed with the road-load model a_m and the commanded torque τ. On the target the state acted on is one request old, so the prediction uses the command before last; only the target exposed this.

$$\hat{v}_k^- = \hat{v}_{k-1} + [a_m(\hat{v}_{k-1}, \tau_{k-j}) + \hat{d}_{k-1}]\Delta t, \qquad \hat{d}_k^- = \hat{d}_{k-1} \tag{5}$$

Here $j = 1$ in the host loop, $j = 2$ on the target, and Δt is 10 ms. Process allowances of $q = 0.002$ m/s and $q_d = 0.02$ m/s² were set on the first 600 s of the urban training cycle. The measurement z_k corrects both states, with a variance r^2 re-estimated from the jitter of the reading.

$$\nu_k = z_k - \hat{v}_k^-, \quad S_k = P_{vv}^- + r^2, \quad \hat{v}_k = \hat{v}_k^- + \frac{P_{vv}^-}{S_k}\nu_k, \quad \hat{d}_k = \hat{d}_k^- + \frac{P_{vd}^-}{S_k}\nu_k \tag{6}$$

Two firmware images differing only in this input ran on the target, on three cycles at the noise floor, a second US06 noise

seed, and US06 at three and ten times the floor. The baseline acts on the raw reading. Table 9 and Fig. 9 give the result.

TABLE 9. Energy on the comparable scale and peak battery current for the baseline loop and the two-state estimator, measured, one run per cell.

Setting	Baseline, Wh/km	Two-state, Wh/km	Peak current, baseline / two-state, A
UDDS, floor	168.10	168.05	118 / 111
HWFET, floor	153.98	153.92	97 / 92
US06, floor	190.79	190.61	284 / 262
US06, floor, second seed	190.98	190.56	295 / 260
US06, 3x noise	191.27	190.67	330 / 273
US06, 10x noise	200.42	190.80	341 / 288

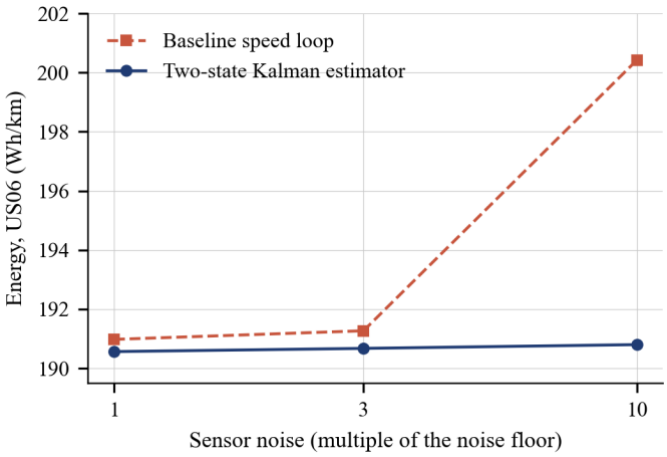


FIGURE 9. Energy on US06 against sensor-noise multiple for the baseline speed loop and the two-state Kalman estimator, measured on the target.

The headline loss is taken against the second US06 floor run. Against it the baseline loop loses 9.44 Wh/km (4.9%) at ten times the noise and the two-state estimator 0.24 Wh/km; against the first floor run the losses are 9.63 and 0.19 Wh/km. The baseline's two floor runs differ by 0.19 Wh/km with only the noise seed changed, so the estimator's residual loss is about the size of run-to-run spread. The peak battery current falls from 341 A to 288 A, the worst control computation grows from 62 to 69 µs of a 10,000 µs period, and no step was late. At the noise floor the arms agree within 0.1% on the three cycles and within 0.22% on the second US06 seed, so no saving is claimed there. The comparison is with the raw-reading loop only; no low-pass filter was run.

One criterion fixed before the run failed. Clean tracking on the urban cycle had to stay within 0.005 m/s of the baseline, and the estimator missed it by 0.004 m/s (0.0193 against 0.0104 m/s). The baseline's own two US06 floor runs differ by 0.011 m/s, so one run per cell cannot resolve the bound; it stays a failure because the bound was set first.

C. FAULT DETECTION FROM AN ACCELERATION RESIDUAL, AND WHY IT FAILED

The first detector compared the speed reading with a speed dead-reckoned from measured acceleration. With an idealised accelerometer (true acceleration plus white noise) it caught 160 of 160 named faults within 0.21 s and raised no false alarm in 3.8 h of clean driving (simulated). On the target it raised one false alarm on a healthy sensor in 1.26 h (measured). The dead-reckoned speed had drifted 0.018 m/s low, and the threshold allowed only for speed noise.

The 70 logged trips (30 h, no sensor fault) show the deeper cause. Logged acceleration and the acceleration implied by logged speed differ by a slow component of about 0.11 m/s², which over the two-second pull of the dead reckoning leaves a standing gap of about 0.22 m/s against a threshold of 0.036 m/s. The detector was flagged 99% of the time. The logs run at 10 Hz and the firmware at 100 Hz, so the flagged share is indicative. The detector is not usable as tuned.

D. BIAS-AWARE FAULT DETECTOR

The bias-aware detector changes the reference the reading is tested against. The dead-reckoned speed carries a state b for the acceleration-channel error, learned with a 2 s time constant and subtracted before integration. A step test handles abrupt faults: a sensor offset arrives in one sample and channel wander does not, so the mean gap $\bar{g}$ of the last $h = 30$ samples is compared with that of the 30 before.

$$|\bar{g}_{\text{new}} - \bar{g}_{\text{old}}| > t_s \sqrt{\frac{2\max(s^2, r^2)}{h}} + \alpha h \Delta t \tag{7}$$

Here s^2 is the pooled variance of the gap, $t_s = 6$, and $\alpha = 0.2$ m/s² allows for a changing channel error. The threshold was tuned on the training cycles only; the cases and criteria were not altered after results were seen.

The simulated cases combine five named faults with six acceleration-channel errors, one of them the slow disagreement replayed from four logged trips per data split, and a vehicle 10% heavier than the model. The detector raised no clean false alarm in 4.5 h on validation (HWFET) or 5.8 h on test (US06) and caught 551 and 533 of 560 named faults within 0.5 s (simulated). It does not meet the criteria fixed for it: the nine late detections on validation are +0.5 m/s offsets at ten times the noise, and the 27 on test were not characterised. The detector was not replayed on the 70 logged trips.

On the target the host injected the faults and simulated the accelerometer, and the detector ran in C, matched to the Python version on 4,000 samples. Table 10 and Fig. 10 give the result for US06 (measured).

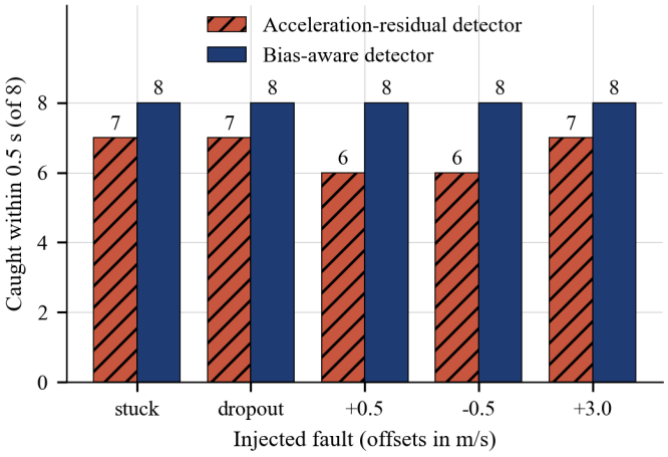


FIGURE 10. Faults caught within 0.5 s in the first campaign on the target, earlier detector against bias-aware detector (measured). Eight faults per type.

In the first campaign the bias-aware detector caught 40 of 40 faults within 0.32 s; the earlier detector caught 33 of 40 and raised 162 false alarms on one clean cycle. The 120 of 120 is these 40 faults repeated over three noise draws, so it counts software faults on a simulated sensor, not independent events. The flag dropped while the fault persisted in 47 of the 120 runs. No false alarm occurred in the one clean hour, which bounds the false-alarm rate below about 3 per hour at 95% confidence. No step was late; the worst control computation was 74 µs.

E. LEARNED REVIEWERS AGAINST A RULE MONITOR

A slower supervisory slot sits around the detector. Every 100 ms it receives twelve summary numbers and may hold a flag, release it, switch the fallback speed, or tighten the tests. It never commands torque. A hand-written rule monitor, a gated recurrent unit (GRU) [42] of 1,508 parameters and a FastGRNN [43] of 550 each filled the slot. Both learned reviewers have 16 hidden states and were trained with AdamW at a learning rate of 3e-3. They ran on the host, on the controller or behind a link with 70 to 1000 ms delay (simulated), and neither was timed on the target.

Scored on diagnosis alone over 540,960 validation blocks, the GRU was the better diagnostician. While the detector's flag was raised, the GRU labelled a real fault correctly in 99% of blocks and the monitor in 94%. While the flag was down, the harder case because the detector had already let the fault go, the GRU still did so in 88% of blocks and the monitor in 1%. In the loop the order reversed (Fig. 11). The monitor cut the worst peak tracking error through a fault from 3.7 to 1.2 m/s on validation and from 5.1 to 1.4 m/s on test. The GRU reached 9.7 m/s on validation and 10.8 m/s on test in its worst cases, the FastGRNN 3.0 and 12.8 m/s, and both raised clean false alarms. Better diagnosis did not give better control. A correct diagnosis keeps the flag up, and both fallback speeds then drift further than the small offset they guard against.

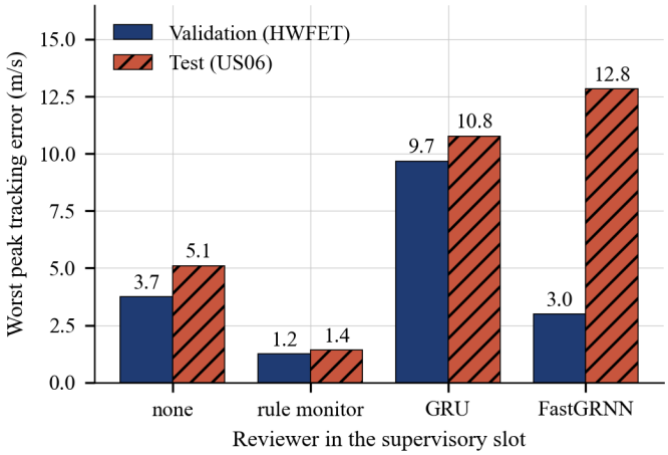


FIGURE 11. Worst peak tracking error through any of 560 named faults for each reviewer on the controller with no delay (simulated). No reviewer means the bias-aware detector alone.

The target corrected the monitor's result. In the first run it looked slightly better than the detector alone (largest error 0.61 against 0.67 m/s), but over three noise draws it was worse, 1.32 against 0.83 m/s, with three alarms left raised where the detector alone left none (measured). No benefit is claimed.

F. FUSED FALLBACK ESTIMATOR

Both fallback speeds drift, one with the acceleration error and

TABLE 10. Injected faults caught within 0.5 s on the target by fault type (measured). First campaign: 8 runs per type. Three noise draws: 24 runs per type.

Fault	Earlier detector, first campaign	Bias-aware, first campaign	Bias-aware, three draws	Worst tracking error, three draws, m/s
Stuck	7 of 8	8 of 8	24 of 24	0.62
Dropout	7 of 8	8 of 8	24 of 24	0.65
Offset +0.5 m/s	6 of 8	8 of 8	24 of 24	0.83
Offset −0.5 m/s	6 of 8	8 of 8	24 of 24	0.72
Offset +3.0 m/s	7 of 8	8 of 8	24 of 24	0.66
All	33 of 40	40 of 40	120 of 120	0.83

one with the model error. The fused fallback estimator is one filter that learns both errors while the sensor is trusted and carries them through a flagged period. In simulation on the validation highway cycle the worst error was 0.62 m/s against 3.74 m/s for the detector alone. On US06 it was best in six of seven conditions, with two alarms left stuck, and worst in the seventh: with the vehicle 10% heavier than the model the worst error was 2.59 m/s, a failure whose cause was not tested. The estimator has not been ported to C or run on the target (simulated only).

VIII. SPEED PLANNING

Everything in this section is simulated on the host computer; nothing ran on the target. The energy managers of Section V could not change the speed trace, and that fixed trace is what left them almost nothing to win. Here the vehicle chooses the trace, so the headroom can be measured where it appears. Eco-driving is a long-standing optimal-control problem [26].

A route is built from a standard drive cycle with the driver removed. It keeps the cycle's distance, its stops and the wait at each, a speed limit at every point, the hardest acceleration and braking the cycle used, and its duration as the arrival time within 1%. The US06 and HWFET routes serve as test and validation sets.

Five arms were compared. The cycle as driven is the reference. A dynamic-programming (DP) planner [20] searches the whole route in 10 m steps, with a price on time set by bisection so that the arrival time is met. The same planner was run with a 300 m horizon, replanning every 40 m, and again with a small network supplying a terminal cost, an estimate of what the rest of the route costs. The fifth arm is a student policy, a network trained on the planner's solutions that names the next speed directly. Every profile was driven through the full vehicle model with the speed loop and the rule-based energy manager, and energy is stated on the comparable scale (Table 11).

TABLE 11. Energy saved against the cycle as driven, by planning arm; one run per route, simulated, no other traffic.

Arm	US06 route (%)	HWFET route (%)
Cycle as driven	0.0	0.0
DP, whole route	10.4	1.2
DP, 300 m horizon	9.5	0.8
300 m, learned terminal cost	10.4	1.1
Student policy	0.03	0.3

On the US06 route the whole-route planner saved 10.4% at the same stops and arrival time. The HWFET route offered 1.2%, since that drive has little to smooth. The terminal cost closed the gap left by the short horizon, and the student policy saved 0.03% on US06.

An empty road is the upper bound. To represent traffic, the planner was held within a corridor around the recorded drive, ahead or behind, at every second, with distance and arrival time unchanged. Fig. 12 shows that the US06 saving was 5.1% within 25 m and 9.7% within 100 m, while HWFET stayed between −0.1% and 1.7%. The corridor planner differs in construction from the empty-road planner, so its 200 m figure of 12.6% is not comparable with the 10.4% above.

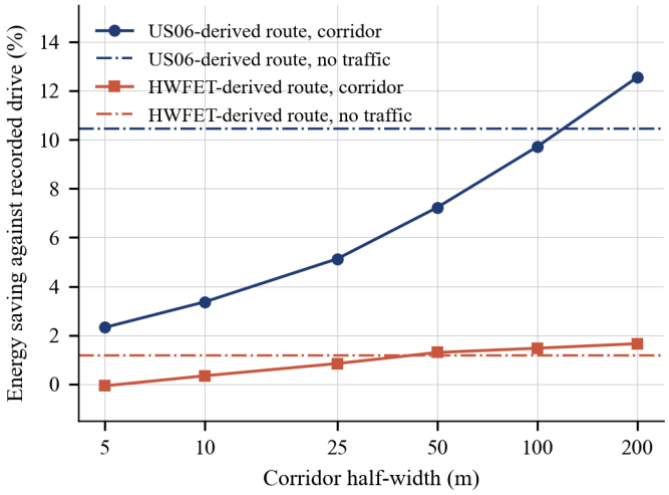


FIGURE 12. Energy saving against corridor half-width, with the no-traffic saving as a dash-dotted reference line for each route. simulated; one run per width.

Planning took 0.19 s in Python on the laptop for all 323 replans of the US06 route with the terminal cost; nothing was ported, so no time or memory on the target is claimed. The traffic is one recorded drive, with no other vehicle, hills or signals, and each arm was run once.

IX. DISCUSSION AND LIMITATIONS

A. WHY THE RULES HELD

The comparison of Section V left little room for any controller. With the speed fixed by the tracking loop, three actuation numbers and cabin heat charged back at the energy it saves, an offline dynamic-programming optimum on the same plant sat within 0.01% to 0.16% of the rules on four cycles. A learned policy had that much to gain and a tracking penalty to pay. Against this bound the corrected campaign behaves as expected: the learned policy used more energy in 11 of 12 distinct conditions, and the one exception lies inside the repeat range of the rule arm.

The null result is therefore about this task. It shows that this SAC policy, tuned once, found nothing on this plant that the rules had missed. It does not show that a policy with authority

over speed could not win; Section VIII shows 5% to 10% on offer once speed is free, for a classical solver.

B. THE AUDIT AS A PROCEDURE

The 8.28% gain was not caught by a gate. Plant gate G2 replays logged trips and never exercised the power-ceiling path, so it passed a model in which a lowered ceiling cut the bill and left the motion alone. The audit began because the campaign tables looked wrong: the learned policy finished below the offline optimum at equal tracking, and its gain grew with demand. That trigger was one-sided. A model error that made learned policies look worse could have passed unexamined, so the checks below should run whatever the direction of the result.

Three checks follow from the case. First, billing and motion must agree: a sweep of constant limits with no controller in the loop should show that a lower ceiling reduces delivered motion as well as the bill (Fig. 5). Second, a learner must not beat the offline optimum computed on the same plant; a result that does is evidence about the plant before it is evidence about the learner. Third, every actuator limit must act on motion and not only on the energy account. None needs the learned policy or the microcontroller. All three run on the host.

Measurement caught other things the host could not. A compute estimate of 22.28 ms for the forecaster became 36.46 ms on the target, 64% higher. The rule monitor's edge in a first run, 0.61 against 0.67 m/s, reversed over three noise draws to 1.32 against 0.83 m/s (measured).

C. WHAT TRANSFERS TO VEHICLE HARDWARE

The ESP32-S3 is a general-purpose microcontroller, not an automotive-qualified part. Some results port by construction: the operation count of a network and its weight memory (19.3 KB for the learned policy, 28,894 B for the timed forecaster graph) do not depend on the vendor. Latency scales with the integer throughput of the target, 29.82 million multiply-accumulates per second here, though an estimate from that scaling was 64% low once.

Timing does not port. The 25 µs period deviation was measured with the target as clock master against a non-real-time host over a serial link, with radio stacks present in the Arduino image though never started. The 8.8 dropped frames per run belong to the host. Flash and cache behaviour, interrupt load from real peripherals and supply current were not examined, nor were the fault-tolerance features of safety-rated parts.

D. SCOPE OF THE SAFETY CLAIMS

The sensor-fault detector, the fallback speeds and the link-outage handling were evaluated as monitoring functions of a research prototype. No claim of conformance with ISO 26262 or the safety of the intended function is made. The fault model is one speed-sensor channel, injected in software, with a simulated accelerometer. The detector has no self-monitoring.

The outage handling shows why the wording matters. On the corrected model, without the handling the worst peak speed error is smaller for outages from 0.2 s to 4 s (3.73 against 6.67 m/s at 4 s). It reverses at 5 s, 7.40 against 7.97 m/s, and at 10 s the handling gives 13.3 against 18.2 m/s (measured). No point between 4 and 5 s was measured, so the handling is described as a behaviour with a stated cost, not as a safety measure.

E. LIMITATIONS

The arrangement is processor-in-the-loop, so nothing here is a hardware-in-the-loop result, and one board and one vehicle model were used. The model's rolling coefficient is 2.2 times a published coastdown value for a different car; it fails gate G2 on 1 of 7 validation trips, missing the state-of-charge bound by 0.45 percentage points (2.45 against 2.00); it was validated on winter trips only; and on the Vehicle Energy Dataset transfer check its median trip energy error is 13.4% to 16.9%. These threaten absolute Wh/km more than the paired ranking on one plant. Yet one plant fault has already changed an answer here. No sensitivity study of the verdict to the rolling coefficient, rotational factor, regeneration ceiling or drivetrain efficiency was run, and the comfort cost of heater shedding is not modelled.

The learned policy is one algorithm, SAC, with one reward design and no sweep. Each campaign cell ran once, except the five repeats of the nominal US06 condition. US06 was held out of training, but it drove the earlier campaign and its fix, so it was not unseen by the authors; the learned tracking deficit appears there (0.17 to 0.50 m/s) and not on UDDS or HWFET (0.004 and 0.016 m/s). Gate G3 fails its zero-dropped-frame criterion; dropped frames and the energy difference are unrelated ($r = -0.03$). The Kalman estimator was compared only with the raw-reading loop. The bias-aware detector was tested on injected faults and not replayed on the logged trips, and the fused fallback estimator, the learned reviewers and the planner are simulated only.

A sensitivity sweep of the plant parameters would show whether the ranking holds. A learned speed planner set against the classical solver would test the one place where headroom exists. A physical accelerometer would replace the simulated drift behind the fault results.

X. CONCLUSION

A processor-in-the-loop campaign on an ESP32-S3 at 100 Hz appeared to show a learned Soft Actor-Critic energy manager

using 8.28% less energy than a rule-based one, in 14 of 16 settings. The saving was an artefact: the vehicle model billed battery power at the policy's ceiling but computed motion from uncapped torque, so a low ceiling cut the bill and left the vehicle driving. A controller-free sweep exposed it, and the gain is withdrawn.

On the corrected model the learned policy used less energy in 1 of 16 settings and 0.94% more on average. That outcome is explained by the task: with speed fixed by the tracking loop, the rules sit within 0.01% to 0.16% of the offline optimum. The finding holds for one vehicle model, one board, one learned algorithm and one reward design.

The method that exposed the fault is the lasting result. A billing-against-motion sweep, a rule that a learner must not beat the offline optimum on the same plant, and a requirement that every limit act on motion would each have caught it before the campaign. The measured gain was in the speed loop, where a two-state Kalman estimator cut the energy lost at ten times sensor noise from 9.44 to 0.24 Wh/km. The detector's 120 of 120 rests on faults injected in software. Planning with free speed offers 5% to 10% in simulation, the place to test a learned method next.

DATA AND CODE AVAILABILITY

The vehicle-model code, firmware images, run logs and analysis scripts used in this study are available from the corresponding author on reasonable request. The BMW i3 trips and the Vehicle Energy Dataset are published datasets, cited in Section IV.

REFERENCES

[1] A. Biswas and A. Emadi, "Energy management systems for electrified powertrains: State-of-the-art review and future trends," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 7, pp. 6453–6467, July 2019, doi: 10.1109/TVT.2019.2914457.

[2] X. Hu, T. Liu, X. Qi, and M. Barth, "Reinforcement learning for hybrid and plug-in hybrid electric vehicle energy management: Recent advances and prospects," *IEEE Industrial Electronics Magazine*, vol. 13, no. 3, pp. 16–25, September 2019, doi: 10.1109/MIE.2019.2913015.

[3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *Proceedings of the 35th International Conference on Machine Learning (PMLR)*, vol. 80, pp. 1861–1870, July 2018.

[4] Espressif Systems, "ESP32-S3 Series Datasheet," *Espressif Systems (datasheet)*.

[5] S. Tangi, A. Vatsa, A. Opam, P. K. Bonthagorla, and D. N. Gaonkar, "Smart strategies for improving electric vehicle battery performance and efficiency," *Scientific Reports*, vol. 15, pp. 42070, November 2025, doi: 10.1038/s41598-025-25987-1.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

[7] M. Steinstraeter, J. Buberger, and D. Trifonov, "Battery and Heating Data in Real Driving Cycles," *IEEE DataPort (dataset)*, October 2020, doi: 10.21227/6jr9-5235.

[8] G. Oh, D. J. LeBlanc, and H. Peng, "Vehicle Energy Dataset (VED), a large-scale dataset for vehicle energy consumption research," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 4, pp. 3302–3312, April 2022, doi: 10.1109/TITS.2020.3035596.

[9] U.S. Environmental Protection Agency, "Dynamometer drive schedules," *U.S. EPA, Vehicle and Fuel Emissions Testing (web resource)*.

[10] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, March 1960, doi: 10.1115/1.3662552.

[11] J. Zhang, Z. Wang, P. Liu, and Z. Zhang, "Energy consumption analysis and prediction of electric vehicles based on real-world driving data," *Applied Energy*, vol. 275, pp. 115408, October 2020, doi: 10.1016/j.apenergy.2020.115408.

[12] M. B. Baluch, S. M. Sharkh, and A. Madhusudhanan, "Electric vehicle energy consumption modeling using real world driving data: System identification vs. machine learning," *IEEE Transactions on Intelligent Vehicles*, vol. 11, no. 3, pp. 401–414, March 2026, doi: 10.1109/TIV.2026.3656694.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 785–794, Aug. 2016, doi: 10.1145/2939672.2939785.

[14] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems 30*, vol. 30, pp. 5998–6008, 2017.

[15] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, February 2019, doi: 10.1016/j.jcp.2018.10.045.

[16] J. Wu, H. He, J. Peng, Y. Li, and Z. Li, "Continuous reinforcement learning of energy management with deep Q network for a power split hybrid electric bus," *Applied Energy*, vol. 222, pp. 799–811, July 2018, doi: 10.1016/j.apenergy.2018.03.104.

[17] W. Sun, Y. Zou, X. Zhang, N. Guo, B. Zhang, and G. Du, "High robustness energy management strategy of hybrid electric vehicle based on improved soft actor-critic deep reinforcement learning," *Energy*, vol. 258, pp. 124806, November 2022, doi: 10.1016/j.energy.2022.124806.

[18] T. Liu, X. Hu, S. E. Li, and D. Cao, "Reinforcement learning optimized look-ahead energy management of a parallel hybrid electric vehicle," *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 4, pp. 1497–1507, August 2017, doi: 10.1109/TMECH.2017.2707338.

[19] R. Xiong, Y. Duan, J. Cao, and Q. Yu, "Battery and ultracapacitor in-the-loop approach to validate a real-time power management method for an all-climate electric vehicle," *Applied Energy*, vol. 217, pp. 153–165, May 2018, doi: 10.1016/j.apenergy.2018.02.128.

[20] R. Bellman, "Dynamic Programming," *Princeton University Press, Princeton, NJ*, 1957.

[21] N. Kim, S. Cha, and H. Peng, "Optimal control of hybrid electric vehicles based on Pontryagin's minimum principle," *IEEE Transactions on Control Systems Technology*, vol. 19, no. 5, pp. 1279–1287, September 2011, doi: 10.1109/TCST.2010.2061232.

[22] G. Paganelli, S. Delprat, T. M. Guerra, J. Rimaux, and J. J. Santin, "Equivalent consumption minimization strategy for parallel hybrid powertrains," *Proceedings of the IEEE 55th Vehicular Technology Conference (VTC Spring 2002)*, vol. 4, pp. 2076–2081, 2002, doi: 10.1109/VTC.2002.1002989.

[23] A. Sciarretta, M. Back, and L. Guzzella, "Optimal control of parallel hybrid electric vehicles," *IEEE Transactions on Control Systems*

Technology, vol. 12, no. 3, pp. 352–363, May 2004, doi: 10.1109/TCST.2004.824312.

[24] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, pp. 3207–3214, April 2018, doi: 10.1609/aaai.v32i1.11694.

[25] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, December 2021.

[26] A. Sciarretta, G. De Nunzio, and L. L. Ojeda, "Optimal ecodriving control: Energy-efficient driving of road vehicles as an optimal control problem," *IEEE Control Systems Magazine*, vol. 35, no. 5, pp. 71–90, October 2015, doi: 10.1109/MCS.2015.2449688.

[27] M. Wegener, L. Koch, M. Eisenbarth, and J. Andert, "Automated eco-driving in urban scenarios using deep reinforcement learning," *Transportation Research Part C: Emerging Technologies*, vol. 126, pp. 102967, 2021, doi: 10.1016/j.trc.2021.102967.

[28] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mane, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, June 2016.

[29] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and characterizing reward gaming," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, December 2022.

[30] J. Lehman, J. Clune, D. Misevic, and et al., "The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities," *Artificial Life*, vol. 26, no. 2, pp. 274–306, 2020, doi: 10.1162/artl_a_00319.

[31] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, September 2017, doi: 10.1109/IROS.2017.8202133.

[32] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," *Proceedings of the IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 737–744, December 2020, doi: 10.1109/SSCI47803.2020.9308468.

[33] R. David et al., "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021.

[34] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," *Advances in Neural Information Processing Systems 33*, vol. 33, 2020.

[35] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, June 2018, doi: 10.1109/CVPR.2018.00286.

[36] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531 (NIPS 2014 Deep Learning and Representation Learning Workshop)*, March 2015, doi: 10.48550/arXiv.1503.02531.

[37] G. Dulac-Arnold et al., "Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis," *Machine Learning*, vol. 110, no. 9, pp. 2419–2468, April 2021, doi: 10.1007/s10994-021-05961-4.

[38] D. Seo and J. Kang, "Dynamic-model-free vehicle velocity estimation using extended Kalman filter with IMU, steering angle, and wheel speed sensors," *Measurement*, vol. 242, pp. 115810, January 2025, doi: 10.1016/j.measurement.2024.115810.

[39] R. Isermann, "Model-based fault-detection and diagnosis: Status and applications," *Annual Reviews in Control*, vol. 29, no. 1, pp. 71–85, 2005, doi: 10.1016/j.arcontrol.2004.12.002.

[40] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "Long short term memory networks for anomaly detection in time series," *Proceedings of the European Symposium on Artificial Neural Networks (ESANN)*, pp. 89–94, April 2015.

[41] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, July 2009, doi: 10.1145/1541880.1541882.

[42] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, 2014, doi: 10.3115/v1/D14-1179.

[43] A. Kusupati, M. Singh, K. Bhatia, A. Kumar, P. Jain, and M. Varma, "FastGRNN: A fast, accurate, stable and tiny kilobyte sized gated recurrent neural network," *Advances in Neural Information Processing Systems 31*, vol. 31, 2018.

[44] Idaho National Laboratory and Intertek, "2013 Nissan Leaf, advanced vehicle testing, baseline testing results," *U.S. Department of Energy, Vehicle Technologies Program (technical report)*.

MANIKANTA GONUGONDLA [Biography and photograph to be supplied.]